VECTOR

MICROSAR Classic Diagnostic Transformer

Technical Reference

Version 4.32.0

Authors	viscfr, visso, vislsa, visgme, jkugler
Status	Released

Document Information

History

Author	Date	Version	Remarks
viscfr	2016-06-23	1.0.0	Initial version
viscfr	2016-11-18	1.1.0	Version update only
vissi	2017-03-20	1.2.0	Version update only
viscfr	2017-06-06	1.3.0	Version update only
viscfr	2017-08-17	1.4.0	Support of AUTOSAR 4.3.0
vissi	2017-03-20	1.5.0	Version update only
visso vissi	2018-03-26	1.6.0	Adapted transformer length parameters according to AUTOSAR 4.3.x Updated referenced documents Added generator to static files
visso	2018-05-07	1.7.0	Reworked chapter 4.1
visso	2018-09-28	1.8.0	Version update only
visso vislsa	2019-04-04	1.9.0	Minor improvements
visso visgme	2019-08-28	1.10.0	Added range checks for data types with limits
visso	2020-04-06	1.11.0	Removed 32-bit build
visso	2020-09-21	1.12.0	Adapted to new template
visso	2021-06-28	1.13.0	Incremented Perl version Added support for MemMap module
visso	2021-11-29	1.14.0	Incremented version
visso	2021-11-29	1.15.0	Updated AUTOSAR version
visgme jkugler	2021-12-22	1.16.0	Added support for float32 data types Product name updated to MICROSAR Classic
visgme	2022-06-30	1.17.0	Version update only
visgme	2023-03-08	4.32.0	Version update only

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	List of Basic Software Modules	R20-11
[2]	Vector	AN-ISC-8-1218_Atomic_Dcm_S-R_Interfaces_with_Diagnostic_Transformer.pdf	

Scope of the Document

This technical reference describes the general use of the MICROSAR Classic Diagnostic Transformer.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2023 Vector Informatik GmbH

Contents

1 Introduction 6
1.1 Architecture Overview 6

2 Functional Description 7
2.1 Features 7
2.2 Initialization 7
2.3 States 7
2.4 Main Functions 7
2.5 Error Handling 7
2.5.1 Development Error Reporting 7
2.5.2 Production Code Error Reporting 7

3 Integration 8
3.1 Embedded Implementation 8

4 API Description 9
4.1 Services provided by DiagXf 9
4.1.1 DiagXf_Init 9
4.1.2 DiagXf_DeInit 9
4.1.3 DiagXf_GetVersionInfo 10
4.1.4 DiagXf_<transformerId> 10
4.1.5 DiagXf_Inv_<transformerId> 11

5 Configuration 12
5.1 Configuration Variants 12

6 Glossary and Abbreviations 13
6.1 Glossary 13
6.2 Abbreviations 13

7 Additional Copyrights 14

8 Contact 15

Illustrations

Figure 1-1 AUTOSAR Architecture Overview 6

Tables

Table 2-1 Supported features 7
Table 3-1 Implementation files 8
Table 4-1 DiagXf_Init 9
Table 4-2 DiagXf_DeInit 9
Table 4-3 DiagXf_GetVersionInfo 10
Table 4-4 DiagXf_<transformerId> 11
Table 4-5 DiagXf_Inv_<transformerId> 11
Table 6-1 Glossary 13
Table 6-2 Abbreviations 13
Table 7-1 Free and Open Source Software Licenses 14

1 Introduction

This document describes the functionality, API and configuration of the MICROSAR Classic BSW module DiagXf.

Supported Configuration Variants:	pre-compile	
Vendor ID:	DIAGXF_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	DIAGXF_MODULE_ID	FF decimal (according to ref. [1])

The DiagXf module provides the functionality to serialize complex data.

1.1 Architecture Overview

The following figure shows where the DiagXf is located in the AUTOSAR architecture.

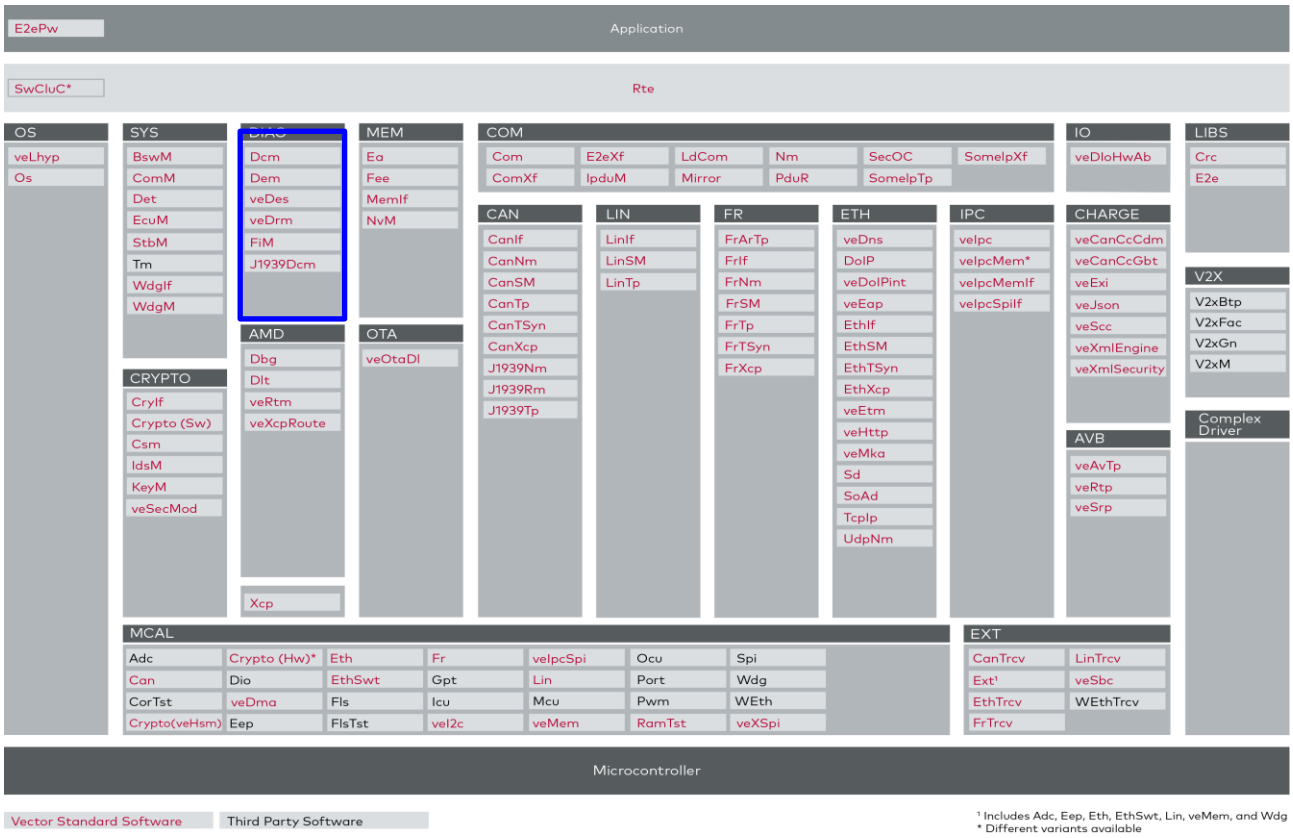


Figure 1-1 AUTOSAR Architecture Overview

2 Functional Description

2.1 Features

The features listed in the following tables cover the complete functionality specified for the DiagXf.

Supported Features
Serialization of diagnostic data elements
Deserialization of diagnostic data elements
Support for uint8, uint16, uint32, sint8, sint16, sint32, boolean record elements
Support for UINT16_N, UINT32_N, SINT8_N, SINT16_N, SINT32_N record elements
Support range checks for data types with data constraints
Support for float32 data type

Table 2-1 Supported features



Caution
DiagXf performs the range checks only when the limits of the float32 type are not set to infinite. DiagXf does not perform special handling for NaN values. If these cause exceptions, the handling needs to be implemented in the application.

2.2 Initialization

The DiagXf does not have to be initialized or deinitialized. Calls to `DiagXf_Init()` and `DiagXf_DeInit()` can be omitted.

2.3 States

No internal states exist.

2.4 Main Functions

No main function exists because all functionality is performed within the called API.

2.5 Error Handling

2.5.1 Development Error Reporting

No development error reporting is currently supported by the DiagXf.

2.5.2 Production Code Error Reporting

No production errors are specified for the DiagXf.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR Classic DiagXf into an application environment of an ECU.

3.1 Embedded Implementation

The delivery of the DiagXf consists of:

File Name	Description	Integration Tasks
DiagXf.c	Generated source file of the DiagXf module.	-
DiagXf.h	Generated main header file which shall be included by modules using the DiagXf module.	-
DiagXf_MemMap.h	Generated file with template areas that can be adapted by the user. It contains the DiagXf specific part of the memory mapping.	Adapt the dedicated code areas within that file. See hints within that file.
DiagXf_Compiler_Cfg.h	Generated file with template areas that can be adapted by the user. It contains the DiagXf specific part of the compiler abstraction.	Adapt the dedicated code areas within that file. See hints within that file.

Table 3-1 Implementation files

4 API Description

4.1 Services provided by DiagXf

4.1.1 DiagXf_Init

Prototype	
`void` **`DiagXf_Init`** `(const DiagXf_ConfigType *config)`	
Parameter	
`config`	Pointer to the transformer's configuration data.
Return code	
`void`	none
Functional Description	
Initialization function.	
Particularities and Limitations	
none	
Expected Caller Context	
This function can be called in any context.	

Table 4-1 DiagXf_Init

4.1.2 DiagXf_DeInit

Prototype	
`void` **`DiagXf_DeInit`** `(void)`	
Parameter	
`void`	none
Return code	
`void`	none
Functional Description	
Deinitialization function.	
Particularities and Limitations	
none	
Expected Caller Context	
This function can be called in any context.	

Table 4-2 DiagXf_DeInit

4.1.3 DiagXf_GetVersionInfo

Prototype	
`void` **`DiagXf_GetVersionInfo`** `(Std_VersionInfoType *versioninfo)`	
Parameter	
`versioninfo`	Pointer to where to store the version information of this module.
Return code	
`void`	none
Functional Description	
This API returns version information, vendor ID and AUTOSAR module ID of the called transformer module.	
Particularities and Limitations	
This API is only available if enabled by the configuration parameter XfrmVersionInfoApi.	
Expected Caller Context	
This function can be called in any context.	

Table 4-3 DiagXf_GetVersionInfo

4.1.4 DiagXf_<transformerId>

Prototype	
`Std_ReturnType` **`DiagXf_<transformerId>`** `(uint8 *buffer, uint32 *bufferLength, const <type> *dataElement)`	
Parameter	
`buffer`	Buffer allocated by the RTE, where the transformed data has to be stored by the transformer.
`bufferLength`	Used length of the buffer.
`dataElement`	Data element which shall be transformed.
Return code	
`E_OK`	Serialization successful.
`DIAGXF_E_SER_OUT_OF_RANGE_ERROR`	`dataElement` value below lower limit and/or exceeds upper limit
Functional Description	
Serialization of the dataElement when communicating from the DataPrototypeMapping.firstDataPrototype to the DataPrototypeMapping.secondDataPrototype.	
Particularities and Limitations	
none	
Expected Caller Context	
This function can be called in any context.	

Table 4-4 DiagXf_<transformerId>

4.1.5 DiagXf_Inv_<transformerId>

Prototype	
`Std_ReturnType` **`DiagXf_Inv_<transformerId>`** `(const uint8 *buffer, uint32 bufferLength, <type> *dataElement)`	
Parameter	
`buffer`	Buffer allocated by the RTE, where the serialized data is stored by the Rte.
`bufferLength`	Used length of the buffer.
`dataElement`	Data element which is the result of the transformation and contains the deserialized data element.
Return code	
`E_OK`	Deserialization successful.
`DIAGXF_E_SER_OUT_OF_RANGE_ERROR`	`dataElement` value below lower limit and/or exceeds upper limit
Functional Description	
Deserialization of the buffer when communicating from the DataPrototypeMapping.secondDataPrototype to the DataPrototypeMapping.firstDataPrototype.	
Particularities and Limitations	
none	
Expected Caller Context	
This function can be called in any context.	

Table 4-5 DiagXf_Inv_<transformerId>

5 Configuration

In the DiagXf the attributes can be configured with the following tools:

> Configuration in DaVinci Configurator

Currently, only the GetVersionInfo API can be enabled / disabled in the DiagXf Ecu configuration.
The serialization / deserialization is based on the `DiagnosticDataElement` described in the DiagnosticExtract. The `BitOffset`, `BaseTypeSize` and `ByteOrder` are considered for each `DiagnosticDataElement`. It is assumed that the `DiagnosticDataElements` are aligned to a byte boundary.
If two incompatible ports are connected using a `DataPrototypeMapping` which references a diagnostic transformer through `firstToSecondDataTransformation`, the DiagXf implementation shall be generated.

5.1 Configuration Variants

The DiagXf supports the configuration variants

> `VARIANT-PRE-COMPILE`

The configuration classes of the DiagXf parameters depend on the supported configuration variants. For their definitions please consider `DiagXf_bswmd.arxml`.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator	Configuration and generation tool for MICROSAR Classic components

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 6-2 Abbreviations

7 Additional Copyrights

The MICROSAR Classic DIAGXF Generator contains *Free and Open Source Software* (FOSS). The following table lists the files which contain this software, the kind and version of the FOSS, the license under which this FOSS is distributed and a reference to a license file which contains the original text of the license terms and conditions. The referenced license files can be found in the directory of the RTE Generator.

File	FOSS	License	License Reference
MicrosarDiagXfGen64.exe	Perl 5.30	Artistic License	License_Artistic.txt

Table 7-1 Free and Open Source Software Licenses

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com